

백엔드 개발자 포트폴리오

{{이름}} · 백엔드 개발자 지원 {{이메일}} · github.com/tmdrud0 · {{블로그 주소}}

한 줄 소개

부하와 장애를 먼저 가정하고, 그 가정이 맞는지 지표로 확인한 뒤 구조를 고치는 개발자입니다.

온라인 저지(Online Judge) 서비스를 혼자 설계·구현하면서, 대회 중 제출이 몰릴 때 데이터가 유실되거나 순위가 어긋나지 않게 만드는 일을 주제로 삼았습니다. "동작하는 코드"가 아니라 "어떤 장애까지 견디는 지 말할 수 있는 코드"를 목표로 했습니다.

핵심 역량

1. 정합성을 순서와 중복에 의존하지 않게 설계합니다 — 3장

채점 완료 순서는 제출 순서와 다릅니다. 이 사실을 무시한 누적 방식 스코어보드는 실시간 순위와 대회 최종 순위를 다르게 만들었습니다. 누적을 버리고 **제출 집합에서 기여도를 매번 다시 계산하는 방식**으로 바뀌, 이벤트 도착 순서와 중복 전달 횟수에 무관하게 같은 결과로 수렴하도록 만들었습니다. 같은 데이터에서 실시간 경로와 DB 재구성 경로가 일치하는 것을 회귀 테스트로 고정했습니다.

2. 추측이 아니라 측정으로 병목을 찾습니다 — 2장

"Redis가 느리다"고 생각했지만, Redis 서버의 `HGET` 실행시간은 $0.72\mu\text{s}$ 였습니다. JFR로 확인한 실제 문제는 **CPU가 포화된 상태에서 동기 호출을 기다리며 park된 Java 스레드**(평균 141ms, 최대 1,240ms)였습니다. 원인을 잘못 짚었다면 Redis를 튜닝했겠지만, 실제 해결책은 Redis를 정확성 경로에서 제거하는 것이었습니다.

3. 성능 수치가 근거가 되지 못할 때를 구분합니다 — 3장 5절

스코어보드 전달 경로를 RabbitMQ Stream으로 바꾸면서 두 구현을 부하 테스트로 비교했지만, 반복 실행이 부족해 성능 차이는 **통계적으로 확인할 수 없다고 판단**했습니다. 채택 근거를 성능이 아니라 **복구 불변식이 단순해진다는 점**으로 명시하고, 남은 검증 항목을 production 전환 게이트로 문서에 남겼습니다.

기술 스택

분류	사용 기술	사용 맥락
언어 · 프레임워크	Java 17, Spring Boot 3.4.3	역할별(web/batch/judge) 프로파일 분리 기동

분류	사용 기술	사용 맥락
데이터 접근	Spring Data JPA, Hibernate, JDBC Template	JPA의 <code>merge</code> 경로 문제를 확인한 뒤 임계 경로는 JDBC batch로 전환
RDBMS	MySQL 8.0	<code>INSERT IGNORE</code> , <code>FOR UPDATE SKIP LOCKED</code> , <code>rewriteBatchedStatements</code>
캐시 · 인메모리	Redis 7	스코어보드 파생 상태, Lua script 원자적 갱신, rate limit
메시징	RabbitMQ 4.1	quorum queue(작업 분배), stream queue(offset 기반 복구), publisher confirm, DLQ
인프라	Docker Compose, Nginx	9개 컨테이너 역할 분리, CPU/메모리 상한 고정
성능 · 관측	Gatling 3.10.5, Prometheus, Grafana, JFR	부하 모델링, 구현 중립 지표 설계, 스레드 park 분석
빌드 · 마이그레이션	Gradle 8.12.1, Flyway	

학력 및 활동

기간	내용
{{기간}}	{{학교 · 전공}}
{{기간}}	{{교육과정 / 스터디 / 수상 등}}

작성한 글

- {{velog 글 제목}} — {{링크}}
- {{velog 글 제목}} — {{링크}}

목차

1. 프로젝트 개요 — 온라인 저지(Online Judge)

- 1.1 무엇을 만들었나
- 1.2 왜 이 구조로 만들었나
- 1.3 시스템 구성
- 1.4 측정 환경
- 1.5 이 포트폴리오에서 다루는 문제

2. 대회 제출 파이프라인 — 저장부터 채점 분배까지

- 2.1 배경 — 만족해야 했던 조건
- 2.2 초기 구조와 한계
- 2.3 저장 경로 — JVM 배치 큐와 batch rewrite
- 2.4 `INSERT IGNORE` — 숨은 누락을 찾아내기
- 2.5 채점 전달 — DB claim에서 outbox + RabbitMQ로
- 2.6 관측 지표로 병목을 다시 찾기
- 2.7 과부하는 어디에 쌓이는가
- 2.8 결과와 남은 한계

3. 스코어보드 전달 경로 — DB Outbox vs RabbitMQ Stream

- 3.1 배경 — 스코어보드는 파생 상태다
- 3.2 문제 — checkpoint를 어디에 둘 것인가
- 3.3 후보 비교
- 3.4 검증 — 두 구현을 실제로 만들어 비교
- 3.5 판단 — 성능이 아니라 복구 불변식을 근거로
- 3.6 남은 한계와 production 전환 게이트
- 3.7 이 장에서 얻은 판단

1. 프로젝트 개요 — 온라인 저지(Online Judge)

항목	내용
기간	{{YYYY.MM ~ YYYY.MM}}
인원	1명 (개인 프로젝트)
저장소	github.com/tmdrud0/{{repo}}
핵심 기술	Java 17, Spring Boot 3.4.3, MySQL 8.0, Redis 7, RabbitMQ 4.1, Docker Compose

1.1 무엇을 만들었나

문제를 풀고 코드를 제출하면 채점 결과와 순위를 돌려주는 온라인 저지 API 서버입니다. 일반 제출도 지원하지만, 이 포트폴리오가 다루는 범위는 **대회(contest) 제출 경로**입니다.

대회 제출은 일반 제출과 부하 특성이 다릅니다.

- 정해진 시간에 수천 명이 동시에 제출합니다.
- 제출이 유실되면 참가자가 대회 중에 알 수 없습니다.
- 순위가 실시간으로 보여야 하지만, 그 순위가 대회 종료 후 최종 순위와 **달라지면 안 됩니다**.

이 세 조건을 동시에 만족시키는 것이 프로젝트의 실제 문제였습니다.

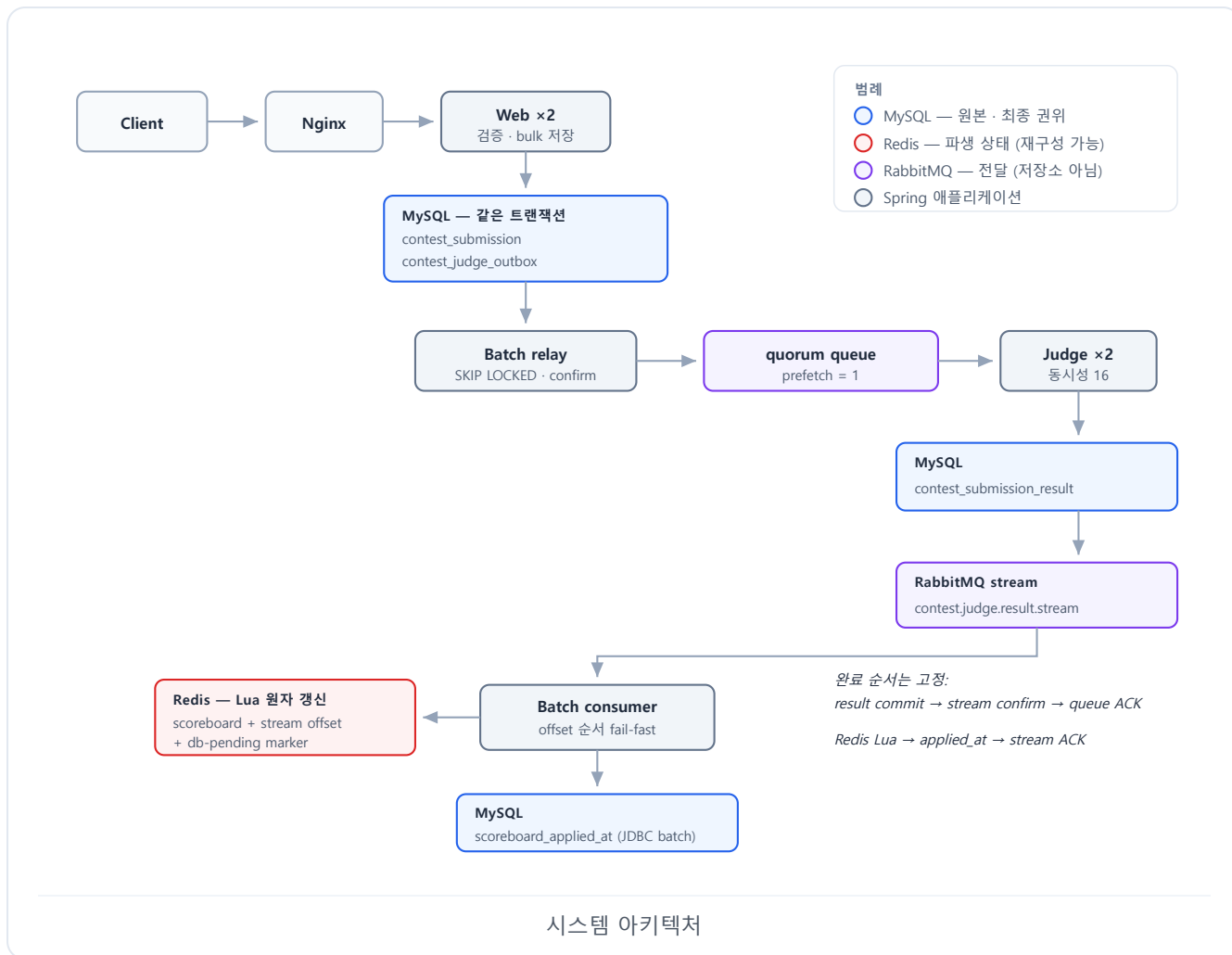
1.2 왜 이 구조로 만들었나

개인 프로젝트인데 인스턴스를 5개로 나눈 이유를 먼저 밝힙니다. 사용자 수 때문이 아니라, 다음을 직접 겪어보기 위한 의도된 구성입니다.

- 프로세스가 죽었을 때 어디까지 복구되는지 **말할 수 있는 시스템**을 만들어보기
- 단일 JVM 안에서는 드러나지 않는 문제(메시지 중복 전달, 이벤트 순서 역전, 인스턴스별 ID 충돌)를 실제로 만나보기
- 병목을 특정 서비스에 귀속시킬 수 있도록 자원 상한을 고정해놓고 측정하기

따라서 이 프로젝트의 성과는 처리량 수치가 아니라 **"어떤 장애를 어떻게 흡수하는지 설명할 수 있게 된 것"**입니다.

1.3 시스템 구성



역할	인스턴스	Spring profile	주 책임
Web	web-1 , web-2	multi-web	JSON API, 제출 검증, 제출 원본과 judge outbox 저장
Batch	batch-1	multi-batch	judge outbox relay, 스코어보드 stream 소비, 랭킹 배치
Judge	judge-1 , judge-2	multi-judge	채점 큐 소비, 채점, 결과 저장, result stream 발행
Data	MySQL, Redis, RabbitMQ	—	원본 / 파생 상태 / durable 전달
Edge	Nginx	—	두 Web 인스턴스 로드밸런싱

저장소별 역할을 명확히 나눴습니다

저장소	역할	유실됐을 때
MySQL	제출 원본, 중복 여부의 최종 권위	복구 불가 — 절대 유실되면 안 되는 유일한 상태
Redis	스코어보드 파생 상태	MySQL 결과에서 언제든지 재구성 가능해야 함
RabbitMQ	작업 전달 수단 (원본 저장소 아님)	재전달 또는 outbox에서 재발행

이 구분이 프로젝트 전체 설계의 기준선입니다. "Redis 스코어보드는 파생 상태다"라는 원칙 하나가 3장의 복구 설계 전체를 결정했습니다.

1.4 측정 환경

절대 수치가 아니라 **같은 전제에서의 상대 비교**를 위해 자원 상한을 고정했습니다.

항목	값
호스트	AMD Ryzen 5 5600 (6C/12T), 16GB RAM, Windows 11
실행	Docker Desktop (WSL2), 12 vCPU / 14GB 예산
애플리케이션 상한 합	7.5 CPU / 9,344M (9개 컨테이너)
관측 스택 상한 합	2.2 CPU / 2,112M (7개 컨테이너)
부하 생성	Gatling 3.10.5 (Windows 호스트에서 별도 실행)

부하 생성기와 서버가 같은 물리 머신에 있으므로 **절대 처리량은 용량 수치로 인용하지 않습니다**. 모든 비교는 같은 상한, 같은 시나리오, 같은 커밋 조건에서 수행했습니다.

1.5 이 포트폴리오에서 다루는 문제

장	주제	핵심 질문
2장	대회 제출 파이프라인	제출 폭주를 유실 없이 받고, 느린 채점 한 건이 전체를 막지 않게 하려면?
3장	스코어보드 전달 경로	Redis가 과거 스냅샷으로 되돌아갔을 때 순위를 어떻게 복구하는가?

두 주제 모두 문제 → 초기 구조의 한계 → 대안 비교 → 선택 근거 → 측정 → 남은 한계 순서로 정리했습니다.

2. 대회 제출 파이프라인 — 저장부터 채점 분배까지

제출 요청 하나가 DB에 저장되고 채점 서버로 전달되기까지의 경로를, 개선 사례별로 나열하지 않고 요청이 지나가는 순서대로 정리했습니다.

2.1 배경 — 만족해야 했던 조건

대회 제출 경로가 지켜야 하는 조건을 먼저 고정했습니다.

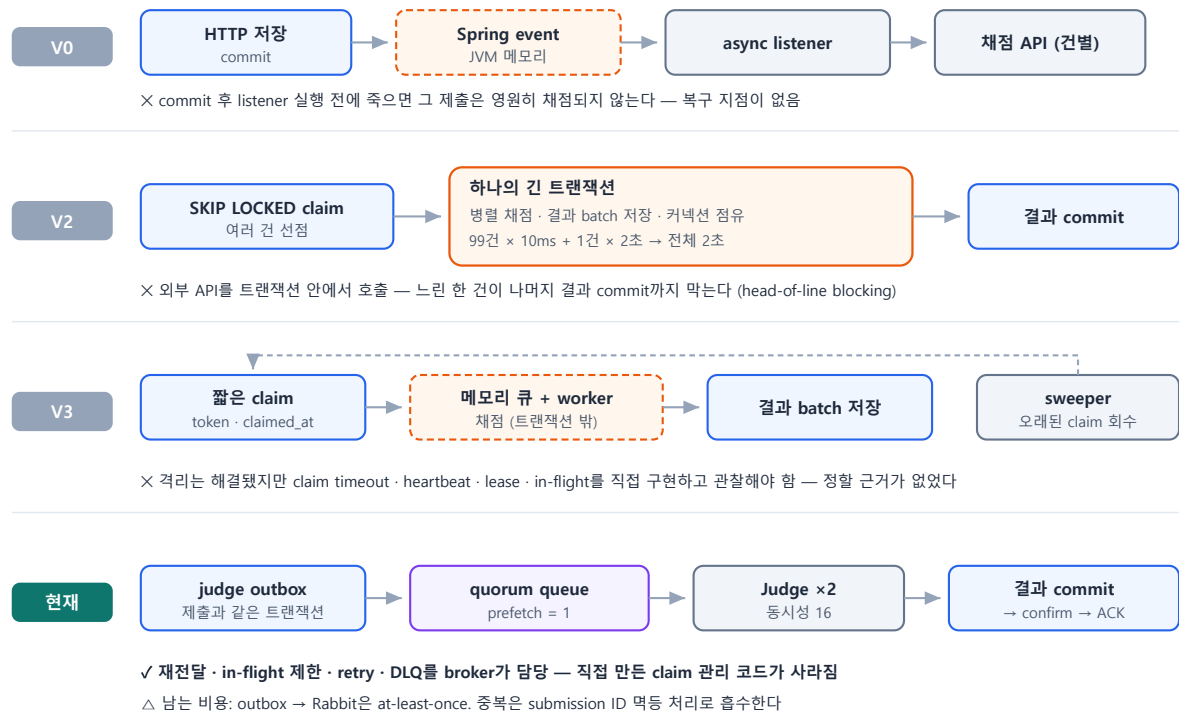
조건	의미
유실 금지	HTTP 성공을 반환했다면 제출 원본은 DB에 commit되어 있어야 한다
중복 저장 금지	(contest, problem, user, codeHash) 가 같은 제출은 한 건만 저장한다
격리	채점이 늦어져도 다른 제출의 처리가 같이 막히지 않아야 한다
복구 가능	프로세스나 broker 장애 후 미완료 작업을 다시 처리할 수 있어야 한다

부하 모델은 다음과 같이 가정했습니다.

최대 유입	약 1,000 TPS
채점 평균	약 10ms (대회용 부분 채점)
채점 긴 꼬리	약 1,000건 중 1건이 최대 2초

이 "1,000건 중 1건"이 설계 전체를 결정한 조건입니다. 평균만 보면 아무 구조나 통과하지만, 2초짜리 한 건이 나머지를 막느냐 아니냐에서 구조가 갈립니다.

2.2 초기 구조와 한계



채점 구조의 변화

V0 — Spring event + 비동기 listener

제출을 저장한 뒤 애플리케이션 이벤트를 발행하고, 비동기 listener가 건별로 채점 API를 호출했습니다.

한계: 이벤트가 JVM 메모리에만 있습니다. DB commit 후 listener 실행 전에 프로세스가 죽으면 **그 제출은 영원히 채점되지 않습니다**. 복구 지점이 어디에도 없었습니다.

V2 — 한 트랜잭션 안에서 여러 건 채점

`SELECT ... FOR UPDATE SKIP LOCKED` 로 미채점 제출을 여러 건 가져와, 같은 트랜잭션 안에서 채점 API를 병렬 호출하고 결과를 batch로 저장했습니다. 중간에 죽으면 rollback으로 정리된다는 점이 매력적이었습니다.

한계: 외부 API 호출 동안 DB 트랜잭션과 커넥션이 계속 열려 있습니다. 그리고 100건을 묶었을 때 **99건이 10ms에 끝나도 1건이 2초면 트랜잭션 전체가 2초입니다**. 2.1절에서 가정한 긴 꼬리가 정확히 이 지점을 때립니다.

V3 — claim / 채점 / 저장 분리

트랜잭션을 짧게 만들기 위해 단계를 나눴습니다.

1. 짧은 트랜잭션으로 DB에서 제출을 claim (PROCESSING, claim_token, claimed_at)
2. 메모리 큐와 worker pool에서 채점
3. 완료 결과를 모아 DB batch로 저장
4. 오래된 claim은 sweeper가 회수

외부 API 호출을 트랜잭션 밖으로 꺼냈고 head-of-line blocking도 사라졌습니다. 하지만 **claim timeout, heartbeat, lease, sweeper, JVM in-flight**를 전부 직접 구현하고 관찰해야 했습니다. timeout이 짧으면 정상적으로 오래 걸린 작업을 중복 채점하고, 길면 실제 장애 복구가 늦어집니다. 이 값을 정할 근거가 없다는 것이 다음 단계로 넘어간 이유입니다.

2.3 저장 경로 — JVM 배치 큐와 batch rewrite

문제: 설정은 batch인데 실제 SQL은 batch가 아니었다

제출마다 즉시 insert하지 않고 JVM 큐에 모아 chunk 단위로 쓰도록 바꿨습니다. Hibernate batch size와 `rewriteBatchedStatements=true`를 켜는데도 처리량이 오르지 않았습니다.

원인은 애플리케이션이 ID를 먼저 발급한 엔티티를 `saveAll()`에 넘긴 것이었습니다. Spring Data JPA는 ID가 이미 있는 엔티티를 신규 row가 아니라고 판단해 `persist`가 아닌 `merge` 경로를 택합니다. 그 결과 신규 제출인데도 row마다 존재 확인 쿼리가 붙었습니다.

```
select ... from contest_submission where id = ?; -- row마다 반복
```

INSERT 앞에 SELECT가 반복되면 batch 설정을 아무리 켜도 요청당 DB 왕복이 줄지 않습니다. 여기에 OSIV가 켜져 있어 커넥션 점유 시간까지 길어졌습니다.

해결

1. DB `AUTO_INCREMENT`를 기다리지 않도록 애플리케이션에서 **Snowflake ID를 먼저 발급**
2. `saveAll()`의 `merge` 대신 `EntityManager.persist()`로 신규 insert 경로를 명확히 함
3. 이후 행별 사후 판별이 필요해지면서 임계 경로를 **JDBC batch로 단일화**
4. **설정값을 믿지 않고** MySQL general log와 드라이버 호출로 실제 multi-values rewrite를 확인

```
INSERT IGNORE INTO contest_submission (...)  
VALUES (...), (...), (...); -- 실제로 이 모양이 되는지 로그로 검증
```

측정 — 저장 경로만 격리했을 때의 첫 실패 지점

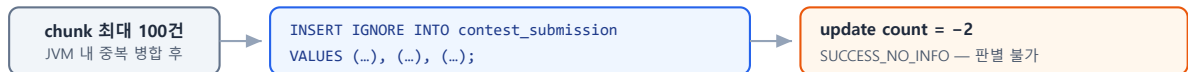
시나리오	Immediate	Bulk
insert 1건, pool 100	4,000 RPS	6,000 RPS
insert 3건, pool 100	4,000 RPS	5,000 RPS
insert 1건, pool 10	2,000 RPS	4,000 RPS
insert 3건, pool 10	2,000 RPS	4,000 RPS

이 수치를 end-to-end 향상률로 읽으면 안 됩니다. 제출 저장 경로만 격리한 값이고, HTTP·RabbitMQ·채점·스코어보드는 포함되지 않았습니다. 커넥션 풀이 작을수록(pool 10) 이득이 큰 것은 batch가 커넥션 점유 시간을 줄이기 때문입니다.

트레이드오프도 있습니다. HTTP 응답이 batch commit을 기다리므로, 부분 chunk는 flush 주기만큼 추가로 대기합니다.

2.4 INSERT IGNORE — 숨은 누락을 찾아내기

① 저장 시도

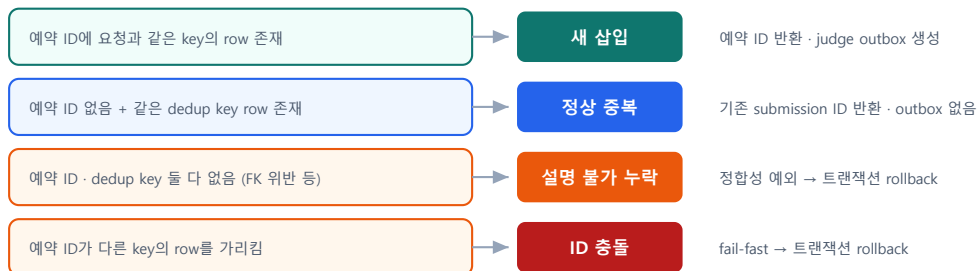


× INSERT IGNORE는 unique 충돌뿐 아니라 FK 위반 같은 오류도 조용히 무시한다 — 무엇이 무시됐는지 드라이버가 알려주지 않는다

② 같은 트랜잭션에서 사후 조회



③ 결과 분류



④ rollback된 chunk를 두 집합으로 나눠 재처리



첫 시도가 rollback으로 완전히 취소됐고 요청별 예약 ID가 고정되어 있으므로 재실행이 안전하다.
재실행에서는 다시 격리하지 않는다 → chunk당 DB 시도 횟수 최대 2회

INSERT IGNORE 사후 판별

문제 1: 중복 한 건이 정상 99건을 함께 rollback시켰다

batch는 최대 100건을 한 트랜잭션으로 묶습니다. 그런데 이미 저장된 코드를 다시 제출한 요청 하나가 unique key를 위반하면, 같은 트랜잭션의 정상 제출 99건까지 전부 rollback됩니다.

일반 INSERT를 INSERT IGNORE로 바꿔 중복 row만 무시하도록 했습니다.

문제 2: INSERT IGNORE는 중복만 무시하는 게 아니었다

INSERT IGNORE는 unique 충돌뿐 아니라 FK 위반 같은 오류도 경고와 함께 조용히 무시합니다. user 사전 SELECT를 제거하고 getReferenceById()로 바꾼 뒤로는 존재하지 않는 user가 FK 위반으로 드러나는데, 이것이 "정상 중복"과 구분되지 않았습니다.

게다가 Connector/J가 batch를 rewrite하면 JDBC update count가 `SUCCESS_NO_INFO(-2)`를 반환합니다. 어떤 행이 삽입되고 어떤 행이 무시됐는지 update count만으로는 알 수 없습니다.

해결: INSERT 직후 같은 트랜잭션에서 다시 조회해 분류

조회 결과	판정	처리
예약 ID에 요청과 같은 key의 row가 존재	새 삽입	예약 ID 반환, judge outbox 생성
예약 ID는 없고 같은 dedup key의 row가 존재	정상 중복	기존 submission ID 반환, outbox 미 생성
예약 ID와 dedup key 모두 없음	설명할 수 없는 누락 (FK 위반 등)	정합성 예외 → rollback
예약 ID가 다른 key의 row를 가리킴	Snowflake ID 충돌 가능성	fail-fast → rollback

이 판별 덕분에 **MySQL이 중복의 유일한 권위**가 되었고, 앞단의 Redis 중복 캐시가 필요 없어졌습니다.

문제 3: 그래도 불량 1건이 정상 99건을 실패시켰다

누락을 발견해도 예외가 발생하면 chunk 전체가 rollback됩니다. 처음에는 그 예외를 chunk 전체의 Future에 그대로 전달해, **존재하지 않는 user 한 명 때문에 정상 제출 99건도 함께 실패**했습니다.

정합성 예외에 문제가 된 예약 ID 목록을 담고, chunk를 두 집합으로 나눴습니다.

```
offenders = 정합성 예외가 지목한 요청 + 같은 dedup key를 공유하는 요청
survivors  = 같은 트랜잭션 때문에 함께 rollback됐을 뿐 정상인 요청
```

offenders의 Future만 실패시키고, survivors는 이미 예약한 동일 ID로 한 번만 다시 batch insert합니다. 첫 시도가 rollback으로 완전히 취소됐고 요청별 예약 ID가 고정되어 있으므로 재실행이 안전합니다. 재실행에서는 다시 격리하지 않아 **chunk당 DB 시도 횟수를 최대 2회로 제한**했습니다.

2.5 채점 전달 — DB claim에서 outbox + RabbitMQ로

2.2절 V3의 claim 구조는 동작했지만, timeout-heartbeat-sweeper를 직접 관리해야 했습니다. 이 책임을 검증된 broker에 넘기기로 하고 후보를 비교했습니다.

후보 비교

후보	장점	이 프로젝트에서의 문제
Kafka classic consumer	높은 순차 처리량, replay, partition 확장	한 번의 <code>poll()</code> 로 받은 record를 worker pool에 넘기면 완료 순서가 offset 순서와 달라짐 . offset 10이 느리고 11~20이 먼저 끝나도 20까지 commit할 수 없어, 연속 구간을 직접 추적해야 함 — claim 코드와 다른 형태의 복잡성이 되돌아옴

후보	장점	이 프로젝트에서의 문제
Kafka Share	record 단위 accept/release, work queue에 가까움	broker가 acquired 상태를 강하게 영속화하지 않아 중복 전달은 여전. 운영·개념 복잡도가 큰데 replay 장점을 이 프로젝트가 쓰지 않음
Redis Streams	이미 Redis를 쓰므로 인프라 추가 없음	pending reclaim, trimming, AOF/RDB durability를 직접 관리. 대량 backlog를 스코어보드와 같은 Redis 메모리에 두는 것이 불리
RabbitMQ quorum queue 	메시지 하나를 consumer 하나에게 주는 work queue 의미가 명확 . prefetch로 consumer별 in-flight 제한. 연속 offset 추적 불필요. retry/DLQ 의미가 작업 처리와 맞음	긴 backlog는 Kafka보다 비쌌음. replay와 다중 consumer group이 자연스럽게 없음

선택 근거: 이 프로젝트의 본질은 replay 가능한 이벤트 로그가 아니라 **각 제출을 한 worker가 가져가 완료하는 work queue**입니다. Kafka가 구조적으로 나쁜 게 아니라, Kafka의 강점(replay·audit·다중 consumer group)을 쓰지 않는 문제였습니다.

왜 RabbitMQ만 쓰지 않았나 (DB outbox를 유지한 이유)

RabbitMQ-only도 검토했지만 다음이 남습니다.

- 제출 코드와 조회 가능한 상태는 **결국 DB에 저장해야 함**
- (contest, problem, user, codeHash) unique 제약은 **DB가 가장 명확하게 보장**
- HTTP 성공 후 DB 저장이 실패하면 사용자가 본 성공과 실제 조회 상태가 달라짐

그래서 HTTP 요청에서 **제출 원본과 contest_judge_outbox**를 같은 트랜잭션에 **commit**합니다. DB와 RabbitMQ 사이에 2PC를 쓰지 않고, outbox와 멍등성으로 at-least-once를 만듭니다.

```
contest_judge_outbox claim (FOR UPDATE SKIP LOCKED, 최대 500건)
```

- persistent message 발행
- Rabbit publisher confirm
- outbox PUBLISHED 완료

주의한 점: outbox가 있다고 해서 Rabbit durability를 낮출 수 있는 게 아닙니다. 현재 relay는 confirm을 받으면 outbox를 PUBLISHED로 바꾸고 **그 이후는 추적하지 않습니다**. 따라서 confirm 이후 유실되면 DB가 자동으로 재발행하지 않으므로, **quorum queue + persistent message + publisher confirm**을 모두 유지해야 합니다.

ACK 순서를 고정했습니다

채점 결과의 완료 순서는 반드시 다음과 같습니다.

```
contest_submission_result commit
→ result stream publisher confirm
→ judge work queue ACK
```

이 순서를 바꾸면 복구 연결이 끊깁니다. publish를 commit 앞에 두거나 confirm을 기다리지 않고 listener를 반환하면, "DB commit 후 publish 전 장애"를 work queue 재전달로 복구할 수 없게 됩니다.

재전달됐을 때는 먼저 `contest_submission_result`를 조회해서, 행이 있으면 **채점 API**를 다시 호출하지 않고 저장된 결과만 재발행합니다.

2.6 관측 지표로 병목을 다시 찾기

구조를 바꾼 뒤 실제 병목이 어디인지 지표로 확인했습니다. 세 번 모두 예상과 달랐습니다.

사례 1 — relay가 33건/초밖에 처리하지 못했다

Rabbit 큐는 계속 비어 있는데 outbox가 쌓였습니다. broker가 아니라 **메시지마다** `markPublished()`를 개별 UPDATE하는 DB 왕복이 병목이었습니다.

개선 전: publish → confirm 대기 → UPDATE (건별 반복)
개선 후: 전부 publish → confirm Future 모아서 대기 → JDBC batch UPDATE 1회

여기서 **relay worker 수를 먼저 늘리지 않은 것**이 판단의 핵심입니다. worker를 늘리면 같은 비효율을 병렬화하면서 DB 커넥션과 lock 경합만 늘렸을 것입니다.

사례 2 — Redis가 느린 게 아니라 스레드가 기다리고 있었다

1,000 TPS 진단 실행에서 응답 p95가 16초까지 올라갔습니다. Redis를 의심했습니다.

측정 대상	값
Redis 서버의 <code>HGET</code> 자체 실행시간	0.72 μs
Redis 서버의 <code>HSET</code> 자체 실행시간	6.62 μs

Redis는 느리지 않았습니다. JFR로 Java 스레드의 park 시간을 봤습니다.

위치	표본 수	평균 park	최대 park
요청 전 dedup <code>HGET</code>	4,641	141.3ms	1,240ms
commit 후 dedup <code>HSET</code>	2,235	61.2ms	1,130ms
TTL 등 추가 Redis 명령	2,207	67.3ms	1,140ms

병목은 Redis의 자료구조 연산이 아니라, **CPU가 포화된 상태에서 Lettuce event loop가 스케줄링을 기다리는 동안 동기 호출을 수행한 Tomcat-completion 스레드가 함께 park된 것**이었습니다.

해결은 Redis 튜닝이 아니라 Redis 제거였습니다. Redis 선조회는 batch 실패 가능성을 낮출 뿐 DB unique 제약을 대체하지 못합니다. 2.4절의 사후 판별이 그 역할을 이미 하고 있었으므로, 요청 전 HGET 과 commit 후 HSET + EXPIRE 를 제출 임계 경로에서 모두 제거했습니다.

사례 3 — 큐는 쌓이지 않았다

같은 진단 실행의 DB bulk 지표입니다.

Web	제출 수	chunk 평균	max pending	completion 큐 평균 대기
Web 1	7,503	36.1ms	101	898ms
Web 2	7,500	36.3ms	90	932ms

MySQL batch insert 큐는 크게 쌓이지 않았고, RabbitMQ:judge·스코어보드도 최종적으로 모두 drain됐습니다. "어딘가 쌓였겠지"라는 추측이 틀렸다는 것을 지표가 알려줬습니다. GC도 최대 194ms pause가 있었지만 초 단위 지연의 주원인이 아니었습니다.

이 진단 실행은 JFR과 스레드 덤프를 켜고 같은 장비에서 부하를 준 것이라 용량 수치로는 쓰지 않고 대기 원인 분석에만 사용했습니다.

2.7 과부하는 어디에 쌓이는가

"큐 하나가 모든 과부하를 흡수한다"는 그림은 이 시스템에 맞지 않습니다. 단계마다 다른 곳에, 다른 연속성으로 쌓입니다.



단계	대기 위치	영속성	포화 동작
DB 저장 전	bulk writer 큐 + semaphore	✗	<code>max-in-flight</code> 초과 시 HTTP 503 + Retry-After
DB 저장 후 응답 전	completion executor (4 threads, queue 64)	✗	큐가 차면 CallerRuns가 bulk worker를 막음
Rabbit publish 전	<code>contest_judge_outbox</code> PENDING rows	✓ MySQL	DB 용량과 relay drain rate만큼 축적
Judge 대기	Rabbit live queue ready messages	✓ quorum queue	broker disk 사용 증가
Judge 처리 중	최대 약 32건 unacked	✓ Rabbit 관리	<code>concurrency=16x2</code> , <code>prefetch=1</code>
스코어보드 대기	stream tail offset – Redis 적용 offset	✓ stream retention	pending offset 증가
최종 실패	Dead-letter queue	✓ quorum queue	운영자가 재처리할 때까지 유지

의도적으로 비영속 구간을 남겼습니다. DB commit 전 요청은 유실될 수 있습니다. 대신 admission semaphore로 유실 가능 범위의 상한을 설정값으로 고정했습니다. `max-in-flight=256` 기준선 측정에서 정확히 상한에서 멈췄고, 거부된 제출 수와 HTTP 503 수가 일치하는 것을 확인했습니다.

`prefetch=1` 을 선택한 이유도 같습니다. consumer 하나가 unacked 메시지를 하나만 갖게 하면, **2초짜리 채점 한 건이 다른 consumer의 진행을 막지 못합니다.** (2.1절의 긴 꼬리 가정)

2.8 결과와 남은 한계

확인한 것

2026-08-09 `submit-100` 실행(제출 19,386건) 기준:

검증 항목	결과
제출 수 = 결과 수 = 스코어보드 반영 수	19,386으로 전부 일치
최종 미반영(unapplied)	0
unacked 상태의 consumer connection을 강제 종료	기존 결과 1행 유지, stream entry만 1건 증가, 채점 API 호출 0회

마지막 항목이 2.5절의 ACK 순서 설계가 실제로 동작한다는 증거입니다.

현재 보장하는 것과 보장하지 않는 것

보장: HTTP 성공 → 제출 원본 DB commit 완료

보장 안 함: DB commit → HTTP 성공 응답

DB commit 후 응답 전에 프로세스가 죽으면 사용자는 실패를 보지만 제출은 존재합니다. 분산 시스템에서 일반적인 결과이며, **클라이언트 재시도 + 멍등 처리로 흡수**합니다. 이때 2.4절의 사후 판별이 기존 submission ID를 돌려주는 것이 중요합니다.

남은 한계

한계	내용
채점 서버가 stub	현재 채점은 즉시 결과를 반환하는 stub입니다. 지금까지의 부하 테스트는 RabbitMQ와 DB 흐름을 검증했지만, 2.1절에서 가정한 p999 2초를 실제로 재현한 것은 아닙니다.
Snowflake worker ID	인스턴스별 유일성을 배포 검증으로 강제하지 못하고 있습니다. 현재는 Compose 설정으로 고정하고, 충돌 시 batch 정합성 조회가 fail-fast합니다.
DLQ 운영	retry를 소진한 메시지는 DLQ로 가지만, 자동 replay 정책은 없습니다.
exactly-once 아님	outbox → Rabbit 경로는 at-least-once입니다. 이를 억지로 exactly-once로 만들지 않고 submission ID 기반 멍등 처리로 중복을 흡수합니다.

이 장에서 얻은 판단

- **캐시 상태를 정확성의 전제로 두지 않는다.** Redis dedup은 성능 최적화였을 뿐 정합성 장치가 아니었고, 결국 임계 경로에서 제거했습니다.
- **설정을 켜다고 동작한다고 믿지 않는다.** batch 설정과 실제 SQL은 달랐고, 로그로 확인해야 알 수 있었습니다.
- **병목을 늘리기 전에 없앤다.** relay worker를 늘리는 대신 건별 DB 왕복을 제거했습니다.

3. 스코어보드 전달 경로 — DB Outbox vs RabbitMQ Stream

실시간 순위를 Redis에 두면 조회는 빨라집니다. 문제는 **Redis가 죽었다 과거 스냅샷으로 살아났을 때 순위를 어떻게 되돌리느냐**입니다. 이 장은 두 가지 전달·복구 구조를 실제로 만들어 비교하고, 무엇을 근거로 선택했는지에 대한 기록입니다.

3.1 배경 — 스코어보드는 파생 상태다

1.3절에서 정한 원칙에서 출발합니다.

MySQL = 원본 (유실되면 복구 불가)
Redis = 파생 상태 (MySQL 결과에서 언제든지 재구성 가능해야 함)

이 원칙이 있으면 "Redis가 죽으면 어떡하지"라는 질문이 "어디서부터 다시 흘려보낼 것인가"라는 훨씬 좁은 질문으로 바뀝니다. 이 장의 전체가 그 재시작 지점(checkpoint)을 어디에 둘 것인가에 대한 이야기입니다.

먼저 해결해야 했던 것 — 채점 완료 순서는 제출 순서와 다르다

복구를 논하기 전에, **재적용이 안전해야 한다**는 전제가 필요했습니다.

기존 스코어보드는 (user, problem) 마다 누적 상태만 저장했습니다.

accepted = 정답 처리 여부
wrongAttempts = 정답 전까지 도착한 오답 수

ACCEPTED가 오면 그 시점의 wrongAttempts로 penalty를 확정하고, 이후 도착한 이벤트는 버렸습니다. **이벤트가 제출 순서대로 도착한다는 숨은 가정이** 있었던 것입니다.

2.1절의 긴 꼬리 가정 때문에 이 가정은 실제로 깨졌습니다.

t=10분 WRONG 제출 → 채점에 2초
t=12분 ACCEPTED 제출 → 채점에 10ms

도착 순서: ACCEPTED → WRONG

경로	계산	결과
실시간 Redis	ACCEPTED 먼저 적용, 늦게 온 WRONG은 버림	$12 + 0 \times 5 = 12$
DB 재구성	제출 시각 순으로 재생	$12 + 1 \times 5 = 17$

대회 중 참가자가 본 순위와 최종 순위가 달라질 수 있었습니다.

해결: 누적하지 않고 제출 집합에서 다시 계산

누적을 버리고, 지금까지 본 제출 사실 자체를 Redis hash에 보존한 뒤 이벤트가 올 때마다 그 문제의 기여도를 처음부터 다시 계산하도록 바꿨습니다.

필드	의미
<code>a:min</code>	가장 이른 ACCEPTED의 contest minute
<code>a:sid</code>	그 ACCEPTED의 submission ID (같은 분 tie-break)
<code>w:<submissionId></code>	각 WRONG 제출의 contest minute
<code>c:solved</code> / <code>c:penalty</code>	이 문제가 summary에 기여 중인 기존 값

새 기여 `penalty` = 가장 이른 ACCEPTED 시각 + (그보다 이른 WRONG 개수 × 5)
`summary` 반영 = 새 기여도 - 기존 `c:*` 기여도 (차이만 HINCRBY)

- 같은 WRONG이 중복 전달되도 `w:<submissionId>` 가 같은 필드를 덮어쓰므로 결과가 변하지 않습니다.
- 더 이른 ACCEPTED가 나중에 도착해도 (분, `submissionId`) 를 비교해 교체하고 다시 계산합니다.

구현 함정 하나: Lua의 number는 double이라 53비트를 넘는 Snowflake ID를 정확히 표현하지 못합니다. Lua script에서는 submission ID를 숫자가 아니라 **길이와 사전순을 이용한 십진 문자열로 비교**합니다.

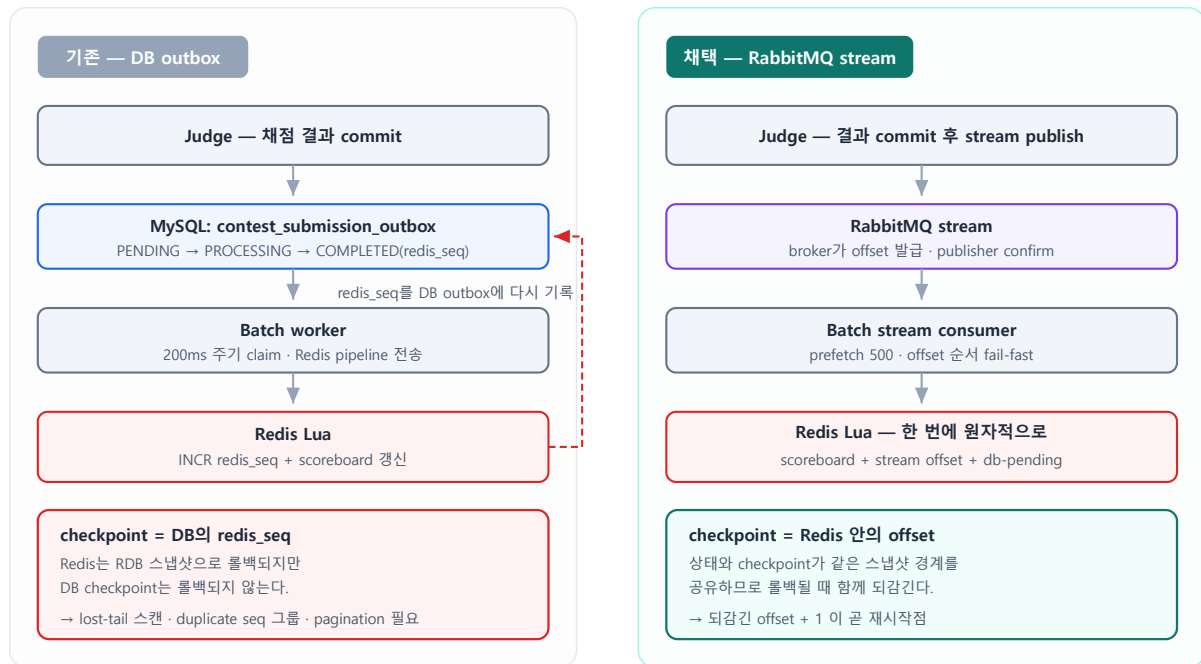
이렇게 해서 **적용 순서와 중복 횟수에 무관하게 같은 결과로 수렴(commutative)**하게 되었고, 비로소 "그냥 다시 흘러보내면 된다"는 복구 전략이 가능해졌습니다.

비용도 있습니다. 오답을 저장하므로 문제별 상태가 커집니다.

문제별 상태	측정 메모리
정답 + wrong 0건	112 bytes
정답 + wrong 20건	696 bytes
정답 + wrong 200건	5,176 bytes

오답 하나당 약 25 bytes가 추가됩니다. 정확성을 위해 메모리를 지불한 선택입니다.

3.2 문제 — checkpoint를 어디에 둘 것인가



핵심은 성능이 아니라 단조 증가 키의 소유권을 누가 갖느냐다.

Redis INCR → DB에 checkpoint ⇒ RabbitMQ offset → Redis 안에 checkpoint

스코어보드 전달 경로 비교

outbox.id 를 checkpoint로 쓸 수 없는 이유

가장 단순한 발상은 "마지막으로 적용한 outbox.id 를 저장해두자"입니다. 하지만:

- DB auto increment ID에는 **gap**이 있을 수 있습니다.
- **DB insert 순서와 Redis 적용 순서가 같지 않습니다.**
- 실패한 작은 ID가 재시도되는 동안 더 큰 ID가 먼저 적용될 수 있습니다.

따라서 `max(outbox.id)` 하나만 저장하면 **중간 누락**을 정확히 판단할 수 없습니다.

근본 원인: 단조 증가 키를 누가 소유하는가

문제를 다시 정의하면 이렇습니다. Redis 상태와 checkpoint가 **같은 시점으로 함께 되감기지 않으면** 복구가 어려워집니다.

Redis 상태 → Redis RDB 스냅샷 시점으로 롤백

checkpoint → DB에 있으므로 롤백되지 않음

↳ 둘의 시점이 어긋남 → 어긋난 구간을 "찾아내야" 함

3.3 후보 비교

후보 A — `redis_seq` 자연 복구 (기존 구현)

Redis Lua가 이벤트 처리마다 전역 `sequence`를 발급하고, 그 값을 DB outbox에 저장합니다. Redis가 RDB로 롤백되면 counter도 낮아지므로 다음 신호가 생깁니다.

- Redis counter보다 큰 DB `redis_seq`가 존재 → **lost tail**
- 롤백 이후 새 이벤트가 과거 `sequence`를 재사용 → **DB에 duplicate `redis_seq`**

복구 worker가 이 신호를 주기적으로 스캔해서 해당 outbox를 `PENDING`으로 되돌립니다.

장점	별도 복구 모드가 필요 없어 운영 흐름을 끊지 않음
단점	복구 속도가 새 이벤트 유입과 스캔 주기에 영향받음. lost-tail 스캔 , duplicate group requeue , pagination 을 정합성 경로에 계속 유지해야 함

후보 B — DB gapless sequence

DB가 gapless sequence를 직접 발급하고, Redis는 그 값을 받아 복구 기준으로 씁니다. 복구 시 Redis가 성공하지 못한 가장 작은 seq를 알려주고 그 지점부터 다시 밀어넣습니다 (go-back-N).

장점	복구에 트래픽이 필요 없음
단점	gapless를 보장하려면 seq 발급 구간을 직렬화 해야 함. 복구를 여러 서버에서 돌리기 어려움

후보 C — RabbitMQ Stream offset

키의 소유권을 뒤집는 방식입니다.

기존: Redis `INCR(redis_seq)` → DB outbox에 checkpoint
변경: RabbitMQ가 offset 발급 → Redis Lua 안에 checkpoint

핵심은 checkpoint를 **Redis 안에** 두는 것입니다. 그러면 스코어보드 상태와 checkpoint가 **같은 RDB 스냅샷 경계를 공유**합니다.

Lua script 하나가 다음 세 가지를 원자적으로 함께 실행합니다.

1. commutative 스코어보드 갱신 (3.1절)
2. `contest:scoreboard:stream:offset` 저장
3. `contest:scoreboard:stream:db-pending`에 submission ID 기록

정상 stored offset N → request N+1 → Lua(상태 + offset N+1) → JDBC applied_at → ACK
Redis 롤백 상태 + offset가 K로 함께 복구 → consumer 재시작 → K+1부터 replay
retention gap 첫 delivery > K+1 → 전체 DB rebuild → 첫 보존 offset을 Lua에서 연결

lost-tail 탐지도, duplicate sequence 그룹도, pagination도 필요 없어집니다. Redis와 함께 되감긴 offset 자체가 재시작점이기 때문입니다.

Redis 적용과 MySQL 쓰기 사이의 틈은 별도로 처리했습니다. 두 저장소가 다르므로

`scoreboard_applied_at` 컬럼은 checkpoint로 쓸 수 없습니다. 대신 Lua가 `db-pending` set을 offset과 함께 기록하고, consumer가 Redis 적용을 마친 뒤 JDBC batch로 완료 처리한 다음 set에서 제거합니다. 그 사이에 죽으면 재전달이나 다음 기동의 `repairPending()` 이 같은 batch update를 반복합니다. SQL은 `COALESCE(scoreboard_applied_at, CURRENT_TIMESTAMP(6))` 라서 최초 완료 시각을 보존합니다.

3.4 검증 — 두 구현을 실제로 만들어 비교

전환 계획을 A/B/C 세 단계로 나눴습니다.

단계	내용	목적
A	기존 outbox를 유지한 채 judge 결과를 stream에 병행 발행	payload가 같은지 먼저 확인 (소비 경로는 그대로)
B	스코어보드 consumer와 checkpoint를 stream으로 전환	실제 구현
C	두 구현을 같은 조건에서 비교 측정	채택 판단

한 번에 갈아엎지 않고 A단계에서 payload 동등성부터 확인한 것이, 문제가 생겼을 때 원인 범위를 좁히는 데 도움이 되었습니다.

측정 조건

outbox 후보: 751b774 (master)
stream 후보: 925f9f3 (A + B단계 포함)

동일 조건: compose.yaml + loadtest + observability, 깨끗한 volume,
submit-100, user 10,000 / problem 5, 같은 자원 상한

각 실행 전에 Prometheus 12/12 target, `oj-app` 5/5, batch-1의 지표 시계열을 두 **scrape** 연속으로 확인한 뒤에야 부하를 시작하도록 했습니다.

이 preflight를 넣게 된 이유: 초기 outbox 실행 5개는 Redis reset 뒤 batch JVM이 Prometheus에 다시 나타나기 전에 부하가 시작된 실행이었습니다. judge outbox가 3,700~5,300건까지 쌓이고 result p95가 34~48초가 되어 **전부 폐기**했습니다. 관측 준비가 안 된 상태의 수치를 결론에 쓰지 않기 위해 harness에 게이트를 추가했습니다.

3.4.1 성능 비교 — 결론을 내리지 않기로 한 이유

`submit-100` 관찰값입니다. `/` 는 1회차 / 2회차입니다.

지표	outbox	stream
scoreboard-applied p50 (ms)	403.3 / 416.7	326.0 / 321.4
scoreboard-applied p95 (ms)	616.6 / 779.5	542.7 / 546.8
scoreboard apply segment p95 (ms)	303.1 / 298.9	167.5 / 156.6
Redis pipeline p95 (ms)	7.90 / 8.62	9.26 / 9.34
scoreboard applied total	19,465 / 19,273	19,419 / 19,419
Redis Lua errors	0 / 0	0 / 0

수치만 보면 stream이 유리해 보입니다. 하지만 **반복 집합 게이트를 확인했습니다.**

후보	result p95 IQR/중앙값	scoreboard p95 IQR/중앙값	판정
outbox	22.005%	11.668%	FAIL (≤ 5% 불충족)
stream	1.395%	0.378%	PASS

원래 계약은 후보별 최소 5회였지만 반복 시간 문제로 2회로 제한된 상태였습니다. **outbox 집합의 실행 간 분산이 너무 커서 성능 비교의 근거로 쓸 수 없다고 판단했고,** end-to-end 성능의 승자를 정하지 않는다고 문서에 명시했습니다.

3.4.2 고부하에서는 오히려 stream이 불리했다

submit-200 (30초 ramp → 200 RPS 120초 유지, 제출 약 26,600건) 단일 관찰입니다.

구간 / 분위	outbox	stream	stream 대 outbox
result-queryable p95	644.4ms	1,917.4ms	+197.5%
scoreboard-applied p95	1,082.6ms	2,520.8ms	+132.8%
scoreboard apply segment p50	206.9ms	136.8ms	-33.9%
scoreboard apply segment p95	475.4ms	947.7ms	+99.3%

낮은 부하에서 보였던 중앙 경로의 이점이 높은 부하의 tail에서는 사라졌습니다.

원인을 한 구간에 귀속시키려 했지만 실패했습니다.

- 스코어보드나 Redis가 관여하기 **전인** result-queryable p95부터 stream이 커졌으므로 Redis 탓이 아닙니다.
- batch-1 에는 judge outbox relay와 stream consumer가 **함께** 있어, stream 실행의 높은 CPU throttling(19.52% vs 11.83%)이 relay에도 퍼졌을 수 있습니다.
- stream은 같은 결과 수에 Redis pipeline을 **2.22배 자주** 호출했습니다 (더 작고 잦은 consumer batch).
- Redis 컨테이너 CPU peak는 오히려 stream이 낮아(13.31% vs 15.48%) Redis 서버 포화로도 설명되지 않습니다.

confirm 전용 지표가 없어 기여도를 분리할 수 없었으므로, "이 요인들이 함께 움직였다"는 범위까지만 결론 냈습니다.

정합성은 두 구현 모두 유지했습니다 — 제출 수 = 결과 수 = 반영 수, 최종 pending 0, DLQ 0, Lua error 0.

3.4.3 복구 실험 — 실제로 Redis를 죽였습니다

이것이 채택을 결정한 측정입니다.

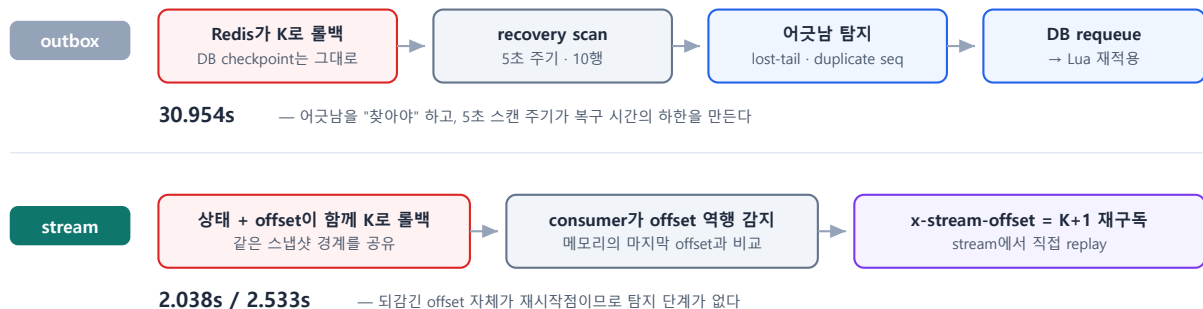
복구 harness의 절차는 다음과 같습니다.

1. batch consumer를 잠시 멈추고 SAVE로 스냅샷 시점 K를 보존
2. 새 제출 tail을 완전히 적용한 시점 N의 스코어보드 SHA-256 digest를 기록
3. Redis를 SIGKILL하고 K의 RDB로 되돌림
4. checkpoint와 전체 스코어보드 digest가 N으로 돌아올 때까지 500ms마다 확인

일부 key만 지우는 시험이 아니라 스코어보드와 checkpoint가 함께 되감기는 일관된 RDB 롤백입니다.

실험 절차 — 일관된 RDB 롤백

- ① batch consumer 정지 → SAVE로 시점 K 보존
 - ② 새 제출 tail을 완전히 적용한 시점 N의 scoreboard SHA-256 digest 기록
 - ③ Redis SIGKILL 후 K의 RDB로 되돌림
 - ④ checkpoint와 digest가 N으로 돌아올 때까지 500ms마다 확인
- 일부 key만 지우는 시험이 아니라 scoreboard와 checkpoint가 함께 되감기는 롤백이다. 롤백된 tail은 약 90건.



이 시간 차이를 성능 배수로 인용하지 않는다

outbox 복구는 시간 절약을 위해 축소된 기준점(참가자 30명 · 73 results)을 사용했고, 이는 outbox에 유리한 조건이다.
stream 두 실험도 의미 상태 수렴 뒤 Docker health 회복 전에 검사한 harness 문제로 최종 게이트가 FAIL이라 수렴 상한 관찰값이다.
드러나는 것은 시간이 아니라 제어 흐름의 복잡도 차이다 — 한쪽은 탐지가 필요하고, 다른 쪽은 필요 없다.

복구 흐름 비교

후보	롤백된 tail	자동 수렴 시간	비고
stream 1	91건	2.533s 이내	digest/pipeline 수렴
stream 2	90건	2.038s 이내	applied +90, rollback counter +1
outbox 1	91건	30.954s	5초 주기 스캔 → DB requeue → Lua 재적용

제어 흐름의 차이가 그대로 드러납니다.

outbox: Redis K → 5초 스캔 → lost-tail/duplicate 탐지 → DB requeue → Lua 재적용
stream: Redis 상태+offset K → 롤백 감지 → K+1부터 stream 직접 replay

이 시간 차이를 성능 배수로 인용하면 안 됩니다. outbox 복구는 시간 절약을 위해 축소된 기준점(30명, 73 results)을 썼고, 이는 **outbox에 유리한 조건**입니다. 또한 stream 두 실행은 의미 상태가 수렴한 뒤 Docker health가 회복되기 전에 검사한 harness 문제로 최종 게이트가 FAIL 처리되어, 정식 PASS 실행의 대표값이 아닌 **수렴 상한 관찰값**으로 취급합니다.

드러나는 것은 시간이 아니라 **제어 흐름의 복잡도 차이**입니다. outbox는 Redis와 DB 사이의 어긋남을 **찾아야** 하고 tail보다 많은 작업을 만들 수 있는 반면, stream은 되감긴 offset 자체가 재시작점입니다.

3.5 판단 — 성능이 아니라 복구 불변식을 근거로

RabbitMQ Stream 방식을 조건부 채택했습니다.

채택 근거를 명확히 구분했습니다.

근거로 쓴 것	근거로 쓰지 않은 것
✅ 복구 불변식이 단순해짐 — lost-tail 스캔, duplicate <code>redis_seq</code> 그룹, pagination, requeue를 운영 정합성 경로에서 제거	❌ end-to-end 성능 (n=2 제한 + outbox 집합 FAIL)
✅ 실제 RDB 롤백에서 offset부터 직접 replay해 같은 스코어보드로 수렴한 관찰	❌ 고부하 처리량 (submit-200에서 오히려 tail 악화)

성능이 좋아서 고른 것이 아닙니다. 오히려 200 RPS에서는 tail 위험을 추가로 관찰했고, 그 사실을 결론과 함께 남겼습니다. 고른 이유는 **운영 중 정합성 경로에서 지워야 할 코드가 줄어든다**는 것입니다.

3.6 남은 한계와 production 전환 게이트

조건부 채택이므로 무엇을 더 검증해야 production에 올릴 수 있는지 명시했습니다.

#	남은 검증	이유
1	judge의 stream confirm 대기시간과 outbox head lag를 분리 계측	3.4.2의 앞단 tail 원인을 confirm 비용과 relay 결합으로 나누지 못함
2	consumer prefetch와 적용 batch 크기 조정 후 CPU throttling 없는 예산에서 재비교	Redis pipeline 호출 수를 outbox 수준으로 맞춰야 공정한 비교
3	후보별 최소 5회 기준 복원	성능을 채택 논거로 쓰려면 필요
4	다중 노드 stream replication / leader failover 검증	현재 Compose는 단일 RabbitMQ 노드. HA 검증 완료로 해석하면 안 됨

구조적으로 남는 한계

- **AMQP 0.9.1에는 stream single-active-consumer 조정이 없습니다.** native stream protocol(5552) 기능이므로, 현재 설계에서는 스코어보드 consumer를 **batch-1** 한 인스턴스로 제한했습니다. Lua가 offset 불연속 시 실패하므로 잘못된 다중 소비가 조용히 offset을 건너뛰지는 않습니다.
- **poison 이벤트를 건너뛰지 않습니다.** Redis pipeline은 앞 명령이 실패해도 뒤 **EVAL** 을 실행할 수 있어 checkpoint가 poison 뒤로 점프할 수 있습니다. 그래서 live batch는 **offset 순서로 fail-fast** 실행하고, batch 전체를 requeue합니다. 스킵보다 정지를 택한 선택입니다.
- **선택적 key 손상은 자동 복구되지 않습니다.** 일관된 RDB 롤백이 아니라 운영자가 일부 key만 지우거나 type이 손상된 경우에는 롤백 신호가 나타나지 않을 수 있습니다. 이 경우 contest 단위 전체 rebuild(`POST /actuator/contestscoreboard?contestId={id}`)가 필요합니다.

검증을 고정한 회귀 테스트

- `InMemoryContestScoreboardCommutativityTests` — 무작위 순서·중복 적용에도 같은 결과
- `RedisContestScoreboardApplierRedisIntegrationTests` — 실제 Redis에서 Lua 계약 검증
- `ContestScoreboardLiveVersusRebuildRedisIntegrationTests` — **실시간 경로와 DB 재구성 경로의 결과 일치**

마지막 테스트가 3.1절에서 발견한 문제(실시간 순위 ≠ 최종 순위)가 재발하지 않는다는 보증입니다.

3.7 이 장에서 얻은 판단

- 복구를 쉽게 만들려면 상태와 checkpoint가 같은 실패 경계를 공유해야 합니다. 둘이 다른 저장소에 있으면 "어긋남을 찾는" 코드가 영원히 필요합니다.
- 재적용이 안전해야 복구 전략을 단순하게 만들 수 있습니다. commutative 갱신(3.1절)이 먼저 있었기 때문에 "그냥 다시 흘려보낸다"가 가능했습니다.
- 측정했는데 결론을 못 내리는 경우가 있습니다. 그때는 수치를 근거로 포장하지 않고, 무엇을 근거로 골랐는지와 무엇이 미검증인지 함께 남기는 편이 낫다고 판단했습니다.